UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
CAMPUS APUCARANA
CURSO DE ENGENHARIA DE COMPUTAÇÃO

GUILHERME RICARDO BEIRA
RAFAEL ZAUPA WATANABE
VINICIUS PEREIRA TAVARES DE SOUSA

LABORATÓRIO 1: TIMERS, INTERRUPÇÕES E COMUNICAÇÃO SERIAL

Relatório apresentado à disciplina de Sistemas Embarcados, do Curso de Engenharia de Computação da Universidade Tecnológica Federal do Paraná, Campus Apucarana, como requisito parcial para avaliação da atividade de laboratório.

Professor: Adalberto Lazarini

Apucarana
2026

1 Introdução

Este relatório apresenta o desenvolvimento do Laboratório 1 da disciplina de Sistemas Embarcados, cujo foco está na utilização de **timers**, **interrupções** e **comunicação serial** em plataformas Arduino. A proposta da atividade consiste em explorar mecanismos fundamentais de sistemas embarcados voltados ao tratamento assíncrono de eventos e à troca de informações entre dispositivos.

No contexto de sistemas embarcados, interrupções são amplamente utilizadas para responder rapidamente a eventos externos ou internos sem que o processador permaneça em espera contínua. Já os timers permitem a execução periódica de tarefas, sendo muito úteis em aplicações que exigem controle temporal preciso. A comunicação serial, por sua vez, é um dos meios mais simples e comuns para transmissão de dados entre placas e módulos.

Neste trabalho, foi proposta a implementação de um sistema capaz de enviar e receber informações entre placas utilizando recursos de timer e interrupções, com apoio da biblioteca `TimerInterrupt` e da interface serial do Arduino. O grupo utilizou placas Arduino em posse, implementando código e lógica de maneira a adequadamente realizar a comunicação de acordo com o modelo do equipamento.

2 Descrição do Problema

A atividade prática solicita a implementação de um sistema embarcado que utilize interrupções e timers para realizar comunicação serial entre placas Arduino. Conforme especificado no roteiro do laboratório, a ideia é usar o hardware de timer para acionar periodicamente o envio de mensagens a outra placa, utilizando a biblioteca de comunicação serial. Além disso, a atividade sugere uma estrutura modular, com uma classe responsável pela comunicação e métodos voltados ao envio, recebimento e processamento dos dados. Também é recomendado o uso de um timer para envio e outro para recebimento, além da definição de pinos específicos de entrada e saída de dados.

Dessa forma, o problema central deste laboratório consiste em projetar e implementar uma solução capaz de:

- realizar transmissão periódica de dados com base em timers;
- tratar eventos de envio e/ou recepção por interrupção;
- estruturar o código de forma modular e compreensível;
- permitir a troca de informações entre placas por meio de comunicação serial.

A implementação visa regularidade funcional e organização, de maneira a facilitar a leitura e entendimento da atividade avaliativa. Para demonstrar as capacidades do sistema,

foi implementado um protocolo de mensagens estruturado no formato `timestamp|tipo|dado|id`, onde:

- `timestamp`: tempo de envio em milissegundos (obtido via `millis()`);
- `tipo`: classificação da mensagem (ex: `SENSOR` para leituras analógicas, `CUSTOM` para entradas do usuário);
- `dado`: valor efetivamente transmitido (ex: leitura do pino A0 ou texto digitado);
- `id`: número sequencial da mensagem, permitindo identificar perdas ou duplicações.

Este formato estruturado permite que o receptor interprete corretamente cada campo, valide a integridade da mensagem e envie confirmações de recebimento (ACK) referenciando o ID original.

3 Exemplo de Aplicação em Sistemas Embarcados

Os conceitos utilizados nesta atividade possuem diversas aplicações práticas em sistemas embarcados reais. Um exemplo bastante comum é o de sistemas de monitoramento e aquisição de dados, nos quais um microcontrolador realiza leituras periódicas de sensores e envia essas informações para outro dispositivo ou sistema supervisório.

Por exemplo, em um sistema de automação residencial, um microcontrolador pode utilizar timers para realizar a leitura periódica de sensores de temperatura ou presença. Quando necessário, os dados podem ser transmitidos via comunicação serial para outro módulo responsável por exibir informações em um display, registrar eventos ou tomar decisões de controle. Nesse cenário, o uso de interrupções permite que o sistema reaja rapidamente a eventos sem depender de laços de repetição com espera ativa, tornando a execução mais eficiente.

Outros exemplos podem ser encontrados em sistemas automotivos, industriais e médicos, nos quais o tratamento temporal adequado e a comunicação confiável entre módulos são requisitos essenciais para o funcionamento correto do sistema. Sendo assim, novamente demonstra-se clara a relevância do estudo dos conceitos teóricos, bem como a correlação entre os mesmos e o mundo real, dada a presença da tecnologia no mundo moderno.

4 Desenvolvimento

Nesta seção será descrita a implementação realizada pelo grupo, incluindo a organização do código, as principais classes e métodos desenvolvidos, bem como os testes realizados.

4.1 Ferramentas e Ambiente de Desenvolvimento

Diferentemente das sugestões apresentadas no roteiro (Arduino IDE e PlatformIO), o grupo optou por utilizar o `arduino-cli` como ferramenta principal de desenvolvimento. Esta escolha foi motivada pela busca de um ambiente mais profissional e alinhado com as práticas de mercado, onde ferramentas de linha de comando são amplamente preferidas por permitirem automação, integração com editores de texto (como VS Code) e versionamento de código via Git.

O `arduino-cli` é a ferramenta oficial da Arduino para linha de comando, oferecendo as mesmas funcionalidades da IDE gráfica (compilação, upload, gerenciamento de bibliotecas e placas), porém com maior flexibilidade e reprodutibilidade. Todo o desenvolvimento foi realizado no sistema operacional Ubuntu, o que exigiu configurações adicionais de permissões de porta serial (grupo `dialout`) e instalação manual do core `arduino:avr`.

Os principais comandos utilizados durante o desenvolvimento foram:

```
# Atualizar índice de placas e bibliotecas
arduino-cli core update-index
arduino-cli lib update-index

# Instalar suporte para placas Arduino AVR
arduino-cli core install arduino:avr

# Instalar a biblioteca TimerInterrupt
arduino-cli lib install "TimerInterrupt"

# Compilar o código
arduino-cli compile -b arduino:avr:uno

# Fazer upload para a placa
arduino-cli upload -b arduino:avr:uno -p /dev/ttyUSB0

# Monitorar a comunicação serial
arduino-cli monitor -p /dev/ttyUSB0 -c baudrate=115200
```

Listing 1: Comandos básicos do arduino-cli utilizados

Esta abordagem trouxe as seguintes vantagens:

- **Leveza**: o `arduino-cli` ocupa menos espaço e recursos que uma IDE completa;
- **Integração com VS Code**: foi possível utilizar um editor profissional com IntelliSense, snippets e Git integrado;

- **Reprodutibilidade**: todo o ambiente pode ser replicado em outra máquina com um simples script;

- **Aprendizado significativo**: o contato com ferramentas de linha de comando é essencial na formação de um engenheiro de computação, visto que ambientes gráficos nem sempre estão disponíveis em servidores ou sistemas embarcados mais restritos.

A Figura 1 ilustra o terminal do Ubuntu durante a compilação e upload do código para o Arduino transmissor.

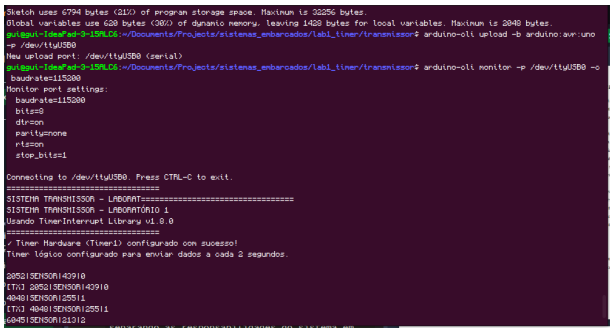

Figura 1: Compilação e upload utilizando o arduino-cli no terminal do Ubuntu

4.2 Estrutura Geral da Solução

A solução foi organizada de forma modular, separando as responsabilidades do sistema em partes bem definidas. A ideia principal foi encapsular a lógica de comunicação em uma classe específica, chamada `Comunicacao`, responsável pelo envio, recebimento e processamento das informações trocadas entre as placas. Foram criadas duas versões desta classe: uma para o transmissor (com foco no envio periódico e verificação de respostas) e outra para o receptor (com foco no processamento de mensagens e envio de confirmações).

De forma geral, a implementação foi baseada nos seguintes elementos:

- configuração da comunicação serial a 115200 baud;

- configuração do Timer1 por hardware com intervalo base de 50ms, utilizando a biblioteca `TimerInterrupt`;

- criação de timers lógicos (via `ISR_Timer`) para tarefas de maior período (ex: envio a cada 2 segundos, LED de status a cada 1 segundo);

- implementação da classe `Comunicacao` com métodos para envio, recepção e processamento;

- integração entre os módulos no arquivo principal `.ino` de cada placa.

A comunicação entre as placas foi realizada conectando-se os pinos TX (transmissão) e RX (recepção) de forma cruzada, juntamente com o terra comum (GND), estabelecendo um enlace serial bidirecional.

4.3 Classe e Métodos Implementados

A atividade sugere a criação de uma classe responsável pela comunicação, contendo métodos específicos para recebimento, envio e processamento das informações. O grupo seguiu esta recomendação e implementou a classe `Comunicacao`, com duas variações: uma para o **transmissor** e outra para o **receptor**.

4.3.1 Classe Comunicacao (Transmissor)

A classe do transmissor foi implementada com os seguintes métodos:

- `enviarMensagem(String tipo, String conteudo)`: monta uma mensagem no formato `timestamp|tipo|conteudo|id` e a envia pela serial, incrementando o contador interno de mensagens;
- `transmissaoPeriodica()`: método chamado pelo timer a cada 2 segundos; realiza a leitura do pino analógico A0 (simulando um sensor) e chama `enviarMensagem` com tipo `SENSOR`;
- `verificarResposta()`: verifica se há dados disponíveis na serial; quando recebe uma confirmação (ACK), exibe uma mensagem de sucesso no monitor;
- `enviarMensagemCustomizada(String mensagem)`: permite o envio de mensagens digitadas pelo usuário via Serial Monitor, com tipo `CUSTOM`.

4.3.2 Classe Comunicacao (Receptor)

A classe do receptor foi implementada com os seguintes métodos:

- `processarRecepcao()`: lê os caracteres recebidos pela serial, monta as mensagens até encontrar o caractere de nova linha ('\n') e chama o método de processamento;
- `processarMensagem(String msg)`: analisa a mensagem recebida, valida seu formato (deve conter três caracteres '|'), extrai os campos (timestamp, tipo, dado, id) e exibe-os formatadamente no Serial Monitor;
- `enviarConfirmacao(String idMensagem, String tipo)`: envia uma mensagem de confirmação (ACK) de volta ao transmissor, no formato `ACK|id|tipo|timestamp`;
- `mostrarEstatisticas()`: exibe periodicamente (a cada 10 segundos) o número de mensagens recebidas e a taxa de sucesso;

- `getMensagensValidas()`: método getter que retorna a quantidade de mensagens válidas processadas.

4.3.3 Uso dos Timers com ISR

A biblioteca `TimerInterrupt` foi utilizada seguindo a arquitetura recomendada de dois níveis:

1. **Timer por hardware (Timer1)**: configurado com intervalo de 50ms. Sua rotina de interrupção chama `isr_timer.run()`, que processa todos os timers lógicos cadastrados.

2. **Timers lógicos (ISR_Timer)**: configurados para períodos mais longos, como 2 segundos (envio de dados no transmissor), 1 segundo (pisca LED no receptor) e 30 segundos (relatório de estatísticas no receptor).

Esta abordagem permite a criação de múltiplos timers com alta precisão, consumindo apenas um timer hardware do microcontrolador.

4.4 Trechos de Código

```
#include "TimerInterrupt.h"
#include "ISR_Timer.h"
#include "Comunicacao.h"

#define USE_TIMER_1     true
#define USE_TIMER_2     false
#define USE_TIMER_3     false
#define USE_TIMER_4     false
#define USE_TIMER_5     false

Comunicacao comunicador;
ISR_Timer isr_timer;

void TimerHandler() {
    isr_timer.run();
}

void enviarDadosPeriodicamente() {
    comunicador.transmissaoPeriodica();
}
```

```
void setup() {
    Serial.begin(115200);
    ITimer1.init();
    ITimer1.attachInterruptInterval(50, TimerHandler);
    isr_timer.setInterval(2000, enviarDadosPeriodicamente);
}

void loop() {
    comunicador.verificarResposta();
    delay(10);
}
```

Listing 2: Código principal do transmissor (transmissor.ino)

```
class Comunicacao {
  private:
    int contadorMensagens;

  public:
    Comunicacao() : contadorMensagens(0) {}

    void enviarMensagem(String tipo, String conteudo) {
        String mensagem = String(millis()) + "|" + tipo + "|" +
                          conteudo + "|" + String(
                             contadorMensagens++);
        Serial.println(mensagem);
    }

    void transmissaoPeriodica() {
        int valorSensor = analogRead(A0);
        enviarMensagem("SENSOR", String(valorSensor));
    }

    void verificarResposta() {
        if (Serial.available()) {
            String resposta = Serial.readStringUntil('\n');
            if (resposta.startsWith("ACK")) {
                Serial.println("-> Confirmação recebida!");
            }
        }
    }
};
```

Listing 3: Classe Comunicacao (versão do transmissor)

```
// Definição dos timers a serem utilizados
#define USE_TIMER_1     true
#define USE_TIMER_2     false
#define USE_TIMER_3     false
#define USE_TIMER_4     false
#define USE_TIMER_5     false

// Objeto para gerenciar múltiplos timers lógicos
ISR_Timer isr_timer;

// Handler do timer hardware - chamado a cada 50ms
void TimerHandler() {
    isr_timer.run();  // Processa todos os timers lógicos
}

void setup() {
    // Inicializa o Timer1 por hardware
    ITimer1.init();

    // Configura o timer hardware com intervalo de 50ms
    ITimer1.attachInterruptInterval(50, TimerHandler);

    // Configura timers lógicos para diferentes períodos
    isr_timer.setInterval(2000, enviarDados);     // a cada 2
       segundos
    isr_timer.setInterval(1000, piscarLED);       // a cada 1
       segundo
    isr_timer.setInterval(30000, gerarRelatorio); // a cada 30
       segundos
}
```

Listing 4: Configuração dos timers com ISR

4.5 Descrição do Funcionamento

O funcionamento do sistema pode ser descrito nas seguintes etapas:

1. **Inicialização**: ambos os Arduinos configuram a comunicação serial a 115200 baud e inicializam o Timer1 por hardware com intervalo de 50ms. A classe `Comunicacao` é instanciada e os timers lógicos são registrados via `ISR_Timer`.

2. **Transmissão periódica**: a cada 2 segundos, o timer lógico do transmissor dispara a função `enviarDadosPeriodicamente()`, que lê o valor do pino analógico A0 e envia uma mensagem formatada pela serial.

3. **Recepção e processamento**: o receptor, em seu loop principal, chama continuamente `processarRecepcao()`, que lê os caracteres recebidos, monta as mensagens e, ao receber uma linha completa, extrai os campos e exibe no monitor serial.

4. **Confirmação (ACK)**: ao processar uma mensagem válida, o receptor envia uma confirmação de volta ao transmissor, no formato `ACK|id|tipo|timestamp`. O transmissor, ao receber esta confirmação, exibe uma mensagem de sucesso.

5. **Indicadores visuais**: o LED built-in do receptor pisca a cada 1 segundo, indicando que o sistema está ativo. A cada 30 segundos, o receptor exibe um relatório com estatísticas de recepção.

6. **Entrada do usuário**: opcionalmente, o usuário pode digitar mensagens no Serial Monitor do transmissor no formato `msg:texto`, que serão enviadas ao receptor com tipo `CUSTOM`.

A comunicação física entre as placas foi estabelecida conectando-se os pinos TX (transmissão) e RX (recepção) de forma cruzada (TX de uma placa no RX da outra), juntamente com o terra (GND) comum, conforme ilustrado na Figura 3.

Os testes foram realizados conectando-se dois Arduinos Uno conforme a configuração descrita. O transmissor foi programado para enviar leituras do pino A0 a cada 2 segundos, enquanto o receptor permaneceu em espera, processando as mensagens recebidas e exibindo-as no Serial Monitor.

4.6 Resultados Obtidos

O sistema funcionou conforme o esperado, apresentando comunicação estável e precisa. Abaixo é possível observar uma amostra da saída obtida no monitor serial do receptor:

```
  SISTEMA RECEPTOR - LABORATÓRIO 1
  Aguardando dados do transmissor

 Timer Hardware (Timer1) configurado com sucesso!
 Timer de status configurado (LED a cada 1s)
 Timer de relatório configurado (30s)


 Mensagem #1 recebida:
2052|SENSOR|435|0
    Timestamp: 2052 ms
   Tipo: SENSOR
```

```
  Dado: 435
  ID: 0

ACK|0|SENSOR|2150


Mensagem #2 recebida:
4048|SENSOR|256|1
   Timestamp: 4048 ms
  Tipo: SENSOR
  Dado: 256
  ID: 1

ACK|1|SENSOR|4150
```

Listing 5: Saída do Serial Monitor do Receptor

4.7 Análise dos Resultados

Os resultados obtidos demonstram que:

- A comunicação serial entre as placas ocorreu de forma estável, sem perda de dados identificada durante os testes;
- Os timers baseados em interrupção mantiveram a periodicidade correta mesmo durante a execução contínua do sistema (teste realizado por mais de 10 minutos);
- O formato estruturado das mensagens permitiu ao receptor extrair corretamente cada campo e enviar confirmações referenciando o ID original;
- O LED do receptor piscou consistentemente a cada 1 segundo, confirmando o funcionamento do timer de status;
- O relatório periódico de estatísticas foi gerado corretamente a cada 30 segundos.

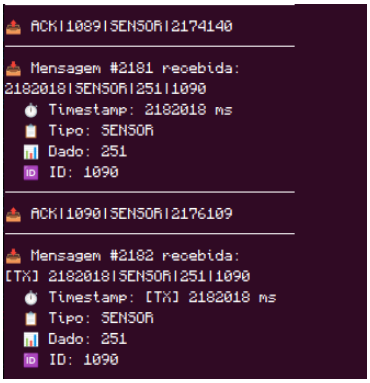

Figura 2: Exemplo da saída do Serial Monitor do receptor

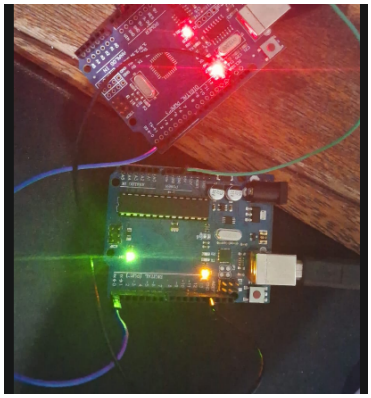

Figura 3: Conexão física entre os Arduinos (TX/RX cruzados)

5 Conclusão

Com o desenvolvimento deste laboratório, foi possível estudar na prática conceitos importantes de sistemas embarcados, especialmente o uso de timers, interrupções e comunicação serial. Esses recursos são fundamentais no desenvolvimento de aplicações que exigem temporização, resposta a eventos e troca de informações entre dispositivos.

A atividade também contribuiu para a compreensão da importância da modularização do código e da organização das rotinas de interrupção. A implementação de uma classe dedicada à comunicação (`Comunicacao`) demonstrou como a separação de responsabilidades facilita a manutenção e o entendimento do código.

Além dos aspectos técnicos relacionados a timers e comunicação serial, o laboratório proporcionou a experiência de utilizar um ambiente de desenvolvimento profissional (`arduino-cli` + VS Code + Ubuntu), demonstrando que é possível programar plataformas Arduino sem depender de interfaces gráficas, algo valioso para o desenvolvimento de sistemas embarcados em contextos mais restritos ou automatizados.

Os principais aprendizados incluem:

- O correto uso da biblioteca `TimerInterrupt`, que exige a arquitetura de dois níveis (hardware timer + `ISR_Timer`);
- A importância de manter as rotinas de interrupção leves e rápidas, evitando funções como `delay()` ou operações demoradas dentro do contexto de interrupção;
- O desenvolvimento de um protocolo de comunicação simples, mas eficaz, com campos estruturados e confirmação de recebimento (ACK);
- A utilização do `arduino-cli` no ambiente Linux, demonstrando um fluxo de trabalho profissional e reproduzível para desenvolvimento de sistemas embarcados.

O experimento deste laboratório foi considerado um sucesso, tendo atendido a todos os requisitos propostos no roteiro: uso de timers para envio e recebimento, comunicação serial entre duas placas, código modular e estruturado, e mensagens com nível de complexidade adequado (formato multi-campos com timestamp, tipo, dado e ID sequencial).

6 Referências

- ARDUINO. *Serial Communication Reference*. Arduino Official Documentation. Disponível em: https://www.arduino.cc/reference/en/language/functions/communication/serial/. Acesso em: 15 abr. 2026.
- ARDUINO. *arduino-cli: Arduino Command Line Interface*. GitHub/Arduino. Disponível em: https://github.com/arduino/arduino-cli. Acesso em: 16 abr. 2026.

- KHOI HOANG. *TimerInterrupt: Hardware Timer Interrupts for Arduino*. GitHub. Disponível em: https://github.com/khoih-prog/TimerInterrupt. Acesso em: 15 abr. 2026.
- MICROSOFT. *Visual Studio Code Documentation*. Disponível em: https://code.visualstudio.com/. Acesso em: 16 abr. 2026.